Tarea 7: Scope, hoisting, arrow functions y closures

Clase 7 | Comportamiento avanzado de funciones

Programa: Propedéutico de JavaScript	**Tiempo estimado:** 45 a 60 minutos
Herramienta: Programiz JavaScript Online Compiler	**Archivo de trabajo:** `main.js`

1. Descripción de la Tarea

En esta tarea se busca que el estudiante comprenda cómo vive una variable según su alcance, qué significa el hoisting y cómo se pueden escribir funciones con sintaxis arrow. Además, se introduce la idea de closure como una forma de conservar estado.

2. Actividades a Desarrollar

Parte A. Ejercicios de refuerzo (Tipo 1)

A1. Crear un ejemplo con variable global, variable local y variable de bloque.

A2. Convertir dos funciones tradicionales a arrow functions.

A3. Probar qué ocurre al intentar usar una variable antes de declararla.

Parte B. Actividad principal aplicada

Desarrollar dos partes. En la primera, crear un contador con funciones para incrementar, disminuir y mostrar valor. En la segunda, crear un closure llamado crearCarrito o crearContador que conserve un valor interno y permita modificarlo sin acceder directamente a la variable desde afuera.

Parte C. Análisis y razonamiento

R1. Explicar en comentarios qué es el scope.

R2. Explicar con palabras propias qué significa hoisting.

R3. Describir por qué un closure puede recordar variables aunque la función principal ya terminó.

3. Entrega y evidencias

- Trabajar todo en Programiz con un único archivo main.js.
- Mostrar resultados que evidencien el cambio de estado en consola.
- Responder la parte analítica dentro del código.

4. Fragmento de referencia

El siguiente bloque es únicamente orientativo. Debe adaptarse a la solución final y no copiarse de manera literal.

```
const multiplicar = (a, b) => a * b;\nconsole.log(multiplicar(4, 5));
```

5. Rúbrica de evaluación (100 puntos)

Rúbrica específica para la clase 7. Se evaluará resolución, claridad y comprensión del tema trabajado.

Criterio	Descripción esperada	Puntaje
Comprensión de scope	El código muestra correctamente distintos alcances de variables.	20
Uso de arrow functions	Las funciones se reescriben correctamente con sintaxis arrow.	15
Closure funcional	La solución principal conserva estado interno de forma correcta.	30
Pruebas en consola	Se evidencia el comportamiento del contador o carrito paso a paso.	20
Explicación conceptual	Los comentarios muestran comprensión de hoisting y closures.	15